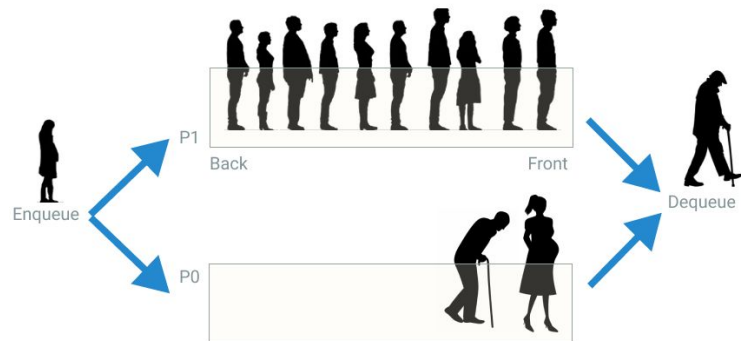


Heap (Priorização)

Prof. Denio Duarte
duarte.denio@uffs.edu.br

Fila de Prioridade

- Existe uma prioridade dos elementos na fila
- Não obedece a estratégia FIFO
- Operações
 - Inserir obedecendo a prioridade
 - Remover o elemento com maior prioridade
- Ideia
 - Organizar a fila colocando o elemento de maior prioridade na frente



Heap

- Uma das estruturas mais eficientes para gerenciar fila de prioridades.
- Estrutura na forma de uma árvore binária
 - Cada **nó interno** (não-folha) tem uma prioridade **maior/menor** ou igual ao dos seus filhos
 - Se escolher menor: **min-heap** (**menor valor** está na raiz)
 - Se escolher maior: **max-heap** (**maior valor** está na raiz)
 - **Não** confundir com a árvore **binária de busca**

Heap

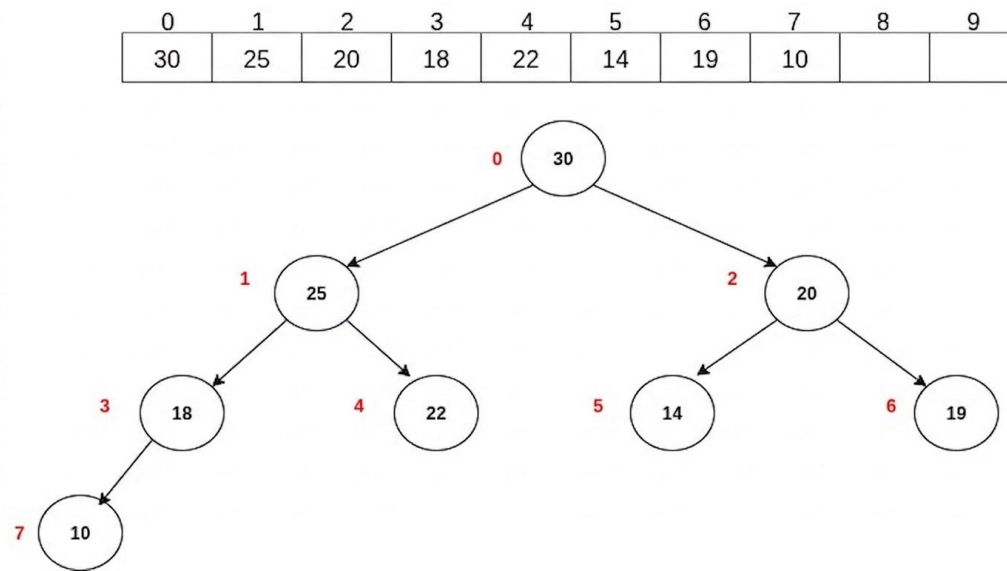
- Pode-se representar como **árvore binária** ou **vetor**
 - Os nós da árvore são numerados para serem representados por um vetor

Heap

- Representação por vetor
 - Linearizar a árvore
 - A raiz é armazenada na primeira posição do vetor
 - O filho à esquerda da raiz na segunda posição
 - O filho à direita da raiz na terceira posição
 - O filho do neto do filho da esquerda da raiz na quarta posição
 - E assim por diante
 - **Conclusão:** o item de maior prioridade (min ou max) estará sempre na raiz da árvore

Heap

- Representação por vetor



Heap

- Vetor

- Cada posição do vetor é considerada pai de outras duas posições
- A posição i passa a ser pai das posições
 - $2i+1 \rightarrow$ filho da esquerda
 - $2i+2 \rightarrow$ filho da direita

Heap

- Vetor

- Cada posição do vetor é considerada pai de outras duas posições
- A posição i passa a ser pai das posições

- $2i+1 \rightarrow$ filho da esquerda

- $2i+2 \rightarrow$ filho da direita

- $(i-1) \div 2$ (int) $\rightarrow$ pai

$i=1 \rightarrow$ **Nó 25**

Filho esquerdo:

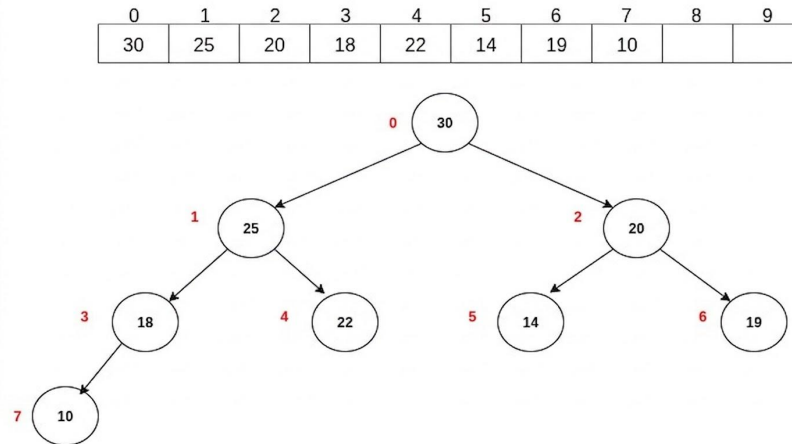
$2 \times 1 + 1 = 3 \rightarrow$ **Nó 18**

Filho direito:

$2 \times 1 + 2 = 4 \rightarrow$ **Nó 22**

Pai:

$(1-1) \div 2 = 0 \rightarrow$ **Nó 30**



Heap

- Estrutura

```
struct TMaxHeap{  
    int heap[SIZE];  
    int size;  
};  
typedef struct TMaxHeap MaxHeap;
```

Heap

- Funções

- Retorna o pai

```
int parent(i) return (int) (i-1)/2
```

- Retorna o irmão da esquerda

```
int left(i) return 2*i+1
```

- Retorna o irmão da direita

```
int right(i) return 2*i+2
```

- **Atenção I:** para quando o **i for igual a zero** (ou seja a **raiz**)

Heap

- Funções

- Retorna o pai

```
int parent(i) return (int) (i-1)/2
```

- Retorna o irmão da esquerda

```
int left(i) return 2*i+1
```

- Retorna o irmão da direita

```
int right(i) return 2*i+2
```

- **Atenção I:** para quando o **i for igual a zero** (ou seja a **raiz**)
- **Atenção II:** a gente guarda o tamanho do heap, assim 0 é vazio

Heap

- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45



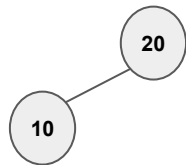
Primeiro elemento size=0 (raiz)

Heap

- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45



Heap

20	10						
----	----	--	--	--	--	--	--

$i = \text{size}$ (antes da inserção)

Pai $i=1$: $(i-1)/2$ pai=0

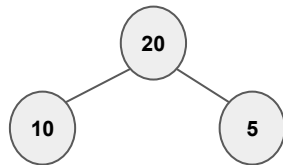
$\text{Heap}[i] > \text{Heap}[\text{pai}]$ (no)

Heap

- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45



Heap

20	10	5					
----	----	---	--	--	--	--	--

$i = \text{size}$ (antes da inserção)

Pai $i=2$: $(2-1)/2$ pai=0

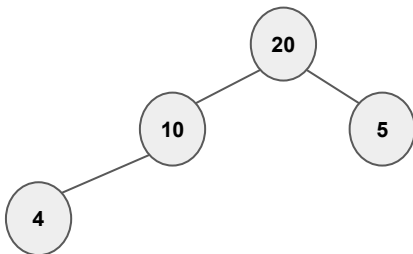
$\text{Heap}[i] > \text{Heap}[\text{pai}]$ (no)

Heap

- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45



Heap

20	10	5	4				
----	----	---	---	--	--	--	--

$i = \text{size}$ (antes da inserção)

Pai $i=3$: $(3-1)/2$ pai=1

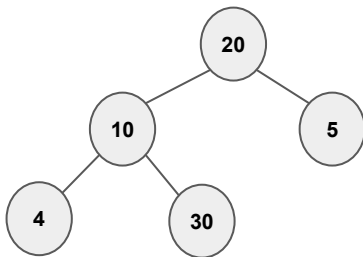
$\text{Heap}[i] > \text{Heap}[\text{pai}]$ (no)

Heap

- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45



Heap

20	10	5	4	30			
----	----	---	---	----	--	--	--

$i = \text{size}$ (antes da inserção)

Pai $i=4$: $(4-1)/2$ pai=1

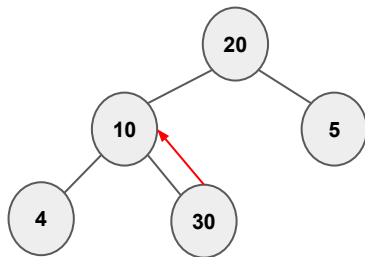
Heap[i] > Heap[pai] (yes)

Heap

- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45



Filho maior que o pai

vetor

20	10	5	4	30			
----	----	---	---	----	--	--	--

$i = \text{size}$ (antes da inserção)

Pai $i=4$: $(4-1)/2$ pai=1

$\text{Heap}[i] > \text{Heap}[\text{pai}]$ (yes)

Troca

$i = \text{pai}$

Heap

- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45

vetor

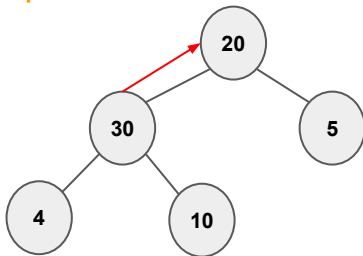
20	30	5	4	10			
----	----	---	---	----	--	--	--

Pai $i=1$: $(1-1)/2$ pai=0

Heap[i] > Heap[pai] (yes)

Troca

Filho maior que o pai



Heap

- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45

vetor

30	20	5	4	10			
----	----	---	---	----	--	--	--

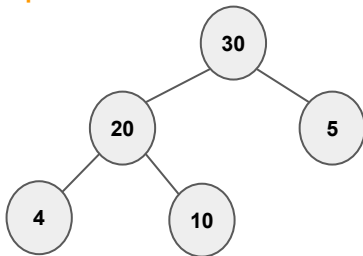
$i = \text{size}$ (antes da inserção)

Pai $i=1$: $(4-1)/2$ pai=0

$\text{Heap}[i] > \text{Heap}[\text{pai}]$ (yes)

Troca

Filho maior que o pai

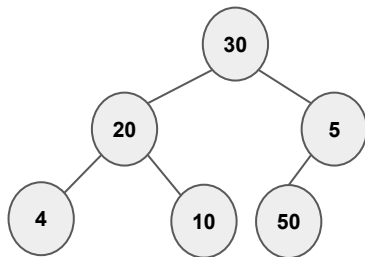


Heap

- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45



vetor

30	20	5	4	10	50		
----	----	---	---	----	----	--	--

$i = \text{size}$ (antes da inserção)

Pai $i=5$: $(5-1)/2$ pai=2

$\text{Heap}[i] > \text{Heap}[\text{pai}]$ (yes)

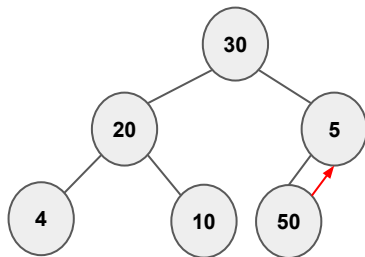
Troca

Heap

- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45



Filho maior que o pai

vetor

30	20	5	4	10	50		
----	----	---	---	----	----	--	--

$i = \text{size}$ (antes da inserção)

Pai $i=5$: $(5-1)/2$ pai=2

$\text{Heap}[i] > \text{Heap}[\text{pai}]$ (yes)

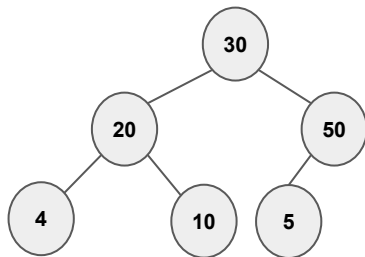
Troca

Heap

- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45



vetor

30	20	5	4	10	50		
----	----	---	---	----	----	--	--

$i = \text{size}$ (antes da inserção)

Pai $i=5$: $(5-1)/2$ pai=2

$\text{Heap}[i] > \text{Heap}[\text{pai}]$ (yes)

Troca

$i = \text{pai}$

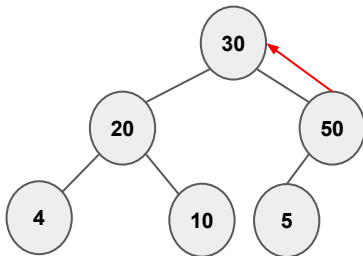
Heap

- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45

Filho maior que o pai



vetor

30	20	50	4	10	5		
----	----	----	---	----	---	--	--

Pai $i=2$: $(2-1)/2$ pai=0

Heap[i] > Heap[pai] (yes)

Troca

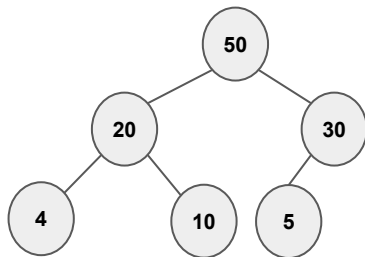
$i=pai$

Heap

- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45



vetor

50	20	30	4	10	5		
----	----	----	---	----	---	--	--

Pai $i=2$: $(2-1)/2$ pai=0

Heap[i] > Heap[pai] (yes)

Troca

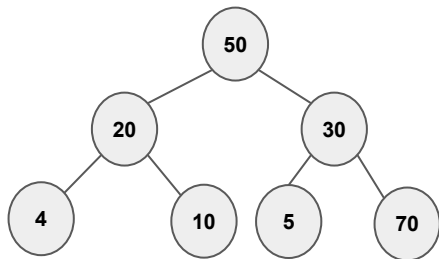
$i=pai$

Heap

- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45



vetor

50	20	30	4	10	5	70	
----	----	----	---	----	---	----	--

$i = \text{size}$ (antes da inserção)

Pai $i=6$: $(6-1)/2$ pai=2

$\text{Heap}[i] > \text{Heap}[\text{pai}]$ (yes)

Troca

$i = \text{pai}$

Heap

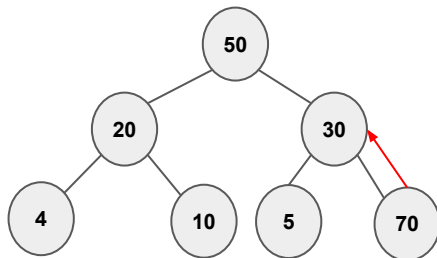
- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45

vetor

50	20	30	4	10	5	70	
----	----	----	---	----	---	----	--



Filho maior que o pai

$i = \text{size}$ (antes da inserção)

Pai $i=6$: $(6-1)/2$ pai=2

$\text{Heap}[i] > \text{Heap}[\text{pai}]$ (yes)

Troca

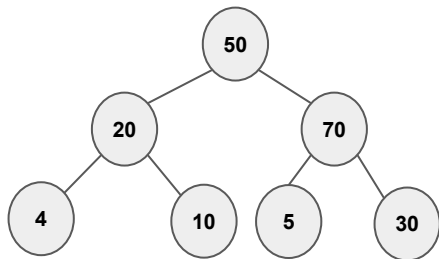
$i = \text{pai}$

Heap

- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45



vetor

50	20	70	4	10	5	30	
----	----	----	---	----	---	----	--

Pai $i=2$: $(2-1)/2$ pai=0

Heap[i] > Heap[pai] (yes)

Troca

$i=pai$

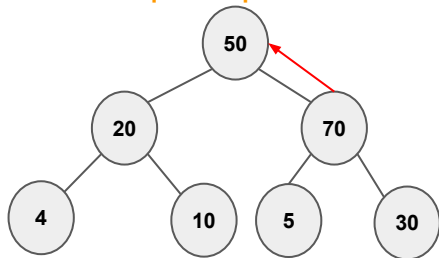
Heap

- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45

Filho maior que o pai



vetor

50	20	70	4	10	5	30	
----	----	----	---	----	---	----	--

Pai $i=2$: $(2-1)/2$ pai=0

Heap[i] > Heap[pai] (yes)

Troca

$i=pai$

Heap

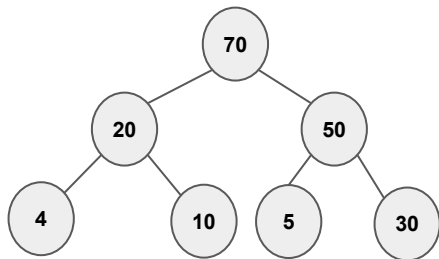
- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45

vetor

70	20	50	4	10	5	30	
----	----	----	---	----	---	----	--



Heap

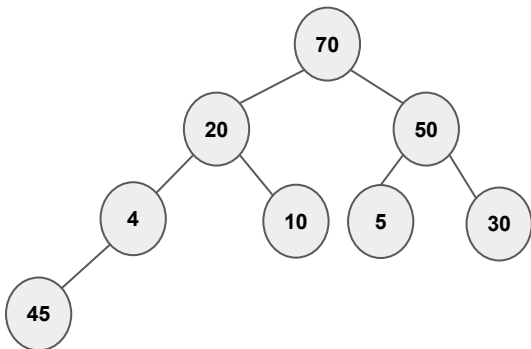
- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45

vetor

70	20	50	4	10	5	30	45
----	----	----	---	----	---	----	----



$i = \text{size}$ (antes da inserção)

Pai $i=7$: $(7-1)/2$ pai=3

$\text{Heap}[i] > \text{Heap}[\text{pai}]$ (yes)

Troca

Heap

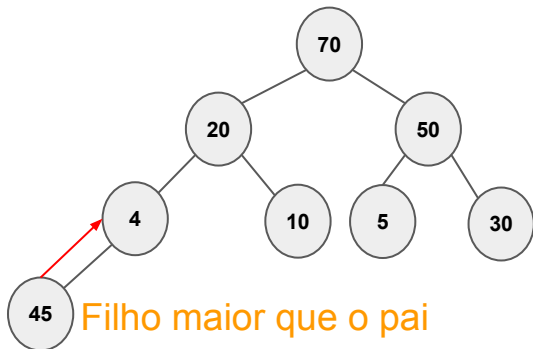
- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45

vetor

70	20	50	4	10	5	30	45
----	----	----	---	----	---	----	----



$i = \text{size}$ (antes da inserção)

Pai $i=7$: $(7-1)/2$ pai=3

$\text{Heap}[i] > \text{Heap}[\text{pai}]$ (yes)

Troca

Heap

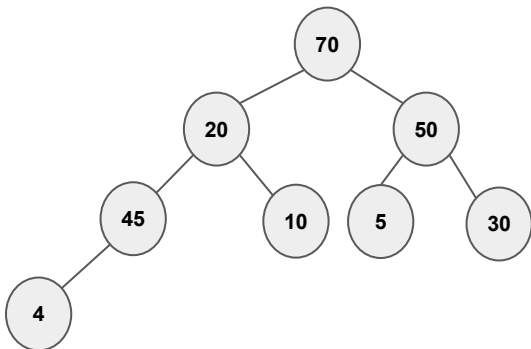
- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45

vetor

70	20	50	45	10	5	30	4
----	----	----	----	----	---	----	---



$i = \text{size}$ (antes da inserção)

Pai $i=7$: $(7-1)/2$ pai=3

$\text{Heap}[i] > \text{Heap}[\text{pai}]$ (yes)

Troca

$i = \text{pai}$

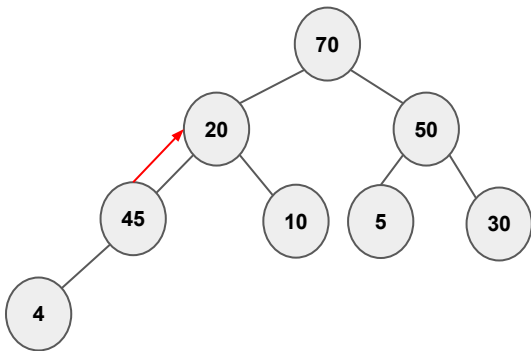
Heap

- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45

Filho maior que o pai



vetor

70	20	50	45	10	5	30	4
----	----	----	----	----	---	----	---

Pai $i=3$: $(3-1)/2$ pai=1

Heap[i] > Heap[pai] (yes)

Troca

Heap

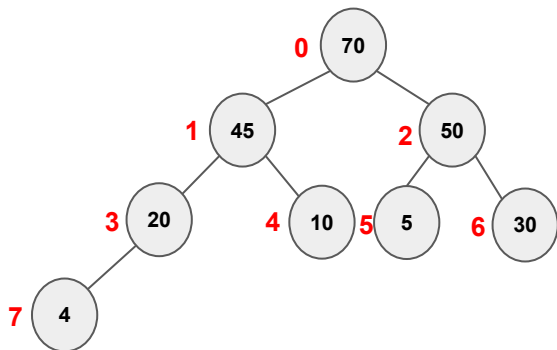
- Inserção (max-heap)

- Coloca o elemento na próxima posição livre do vetor
- Se o pai for menor, sobe
- Sobe até encontrar um pai maior ou chegou na raiz

20, 10, 5, 4, 30, 50, 70, 45

vetor

70	45	50	20	10	5	30	4
----	----	----	----	----	---	----	---



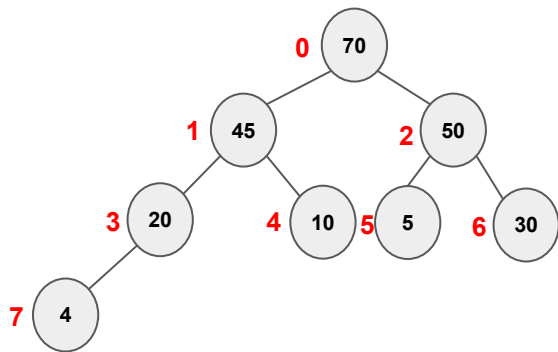
Heap

- Remoção (max-heap)
 - Sempre da raiz
 - Promove o último elemento do vetor como raiz
 - **Rebaixa** o nó trocando com o **maior** dos seus filhos (**caso exista**)

Heap

- Remoção (max-heap)

- Sempre da raiz
- Promove o último elemento do vetor como raiz
- **Rebaixa** o nó trocando com o **maior** dos seus filhos (**caso exista**)



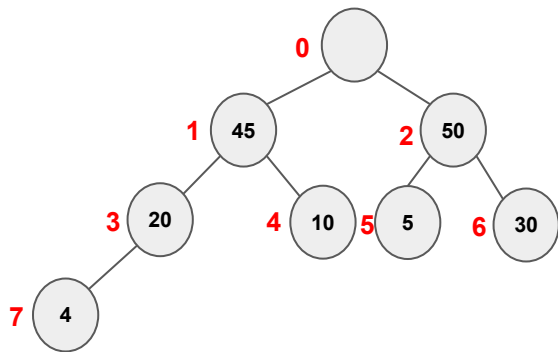
70	45	50	20	10	5	30	4
----	----	----	----	----	---	----	---

Remove() → 70

Heap

- Remoção (max-heap)

- Sempre da raiz
- Promove o último elemento do vetor como raiz
- Realoca o elemento em um nó em que ele seja menor que o filho

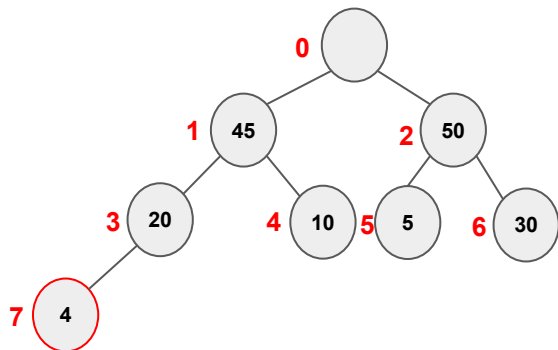


	45	50	20	10	5	30	4
--	----	----	----	----	---	----	---

Remove() → 70

Heap

- Remoção (max-heap)
 - Sempre da raiz
 - Promove o último elemento do vetor como raiz
 - Realoca o elemento em um nó em que ele seja menor que o filho



	45	50	20	10	5	30	4
--	----	----	----	----	---	----	---

Remove() → 70

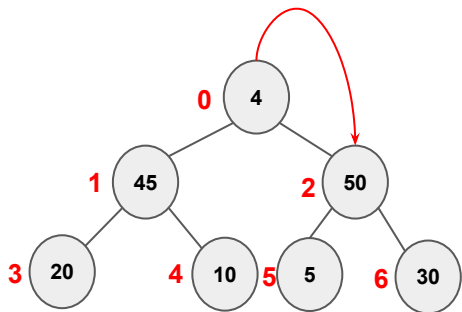
Raiz = o maior filho (size - 1);

Size = size - 1;

Heap

- Remoção (max-heap)

- Sempre da raiz
- Promove o último elemento do vetor como raiz
- Realoca o elemento em um nó em que ele seja menor que o filho



4	45	50	20	10	5	30	4
---	----	----	----	----	---	----	---

Remove() → 70

4 nova raiz

o maior é o filho da direita → troca

$i=0$ (índice da raiz)

$esquerda = 2 * i + 1$

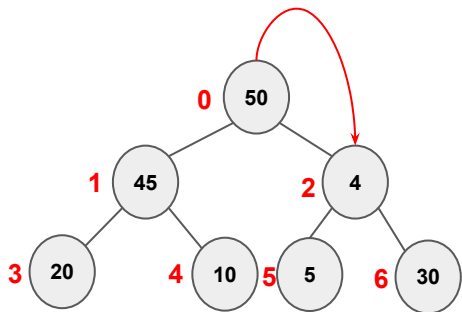
$direita = 2 * i + 2$

Verifica qual dos dois a nova raiz é menor
(esquerda tem preferência)

Heap

- Remoção (max-heap)

- Sempre da raiz
- Promove o último elemento do vetor como raiz
- Realoca o elemento em um nó em que ele seja menor que o filho



4	45	50	20	10	5	30	4
---	----	----	----	----	---	----	---

Remove() → 70

4 nova raiz

o maior é o filho da direita → troca

i=2

Escolhe esquerda como maior

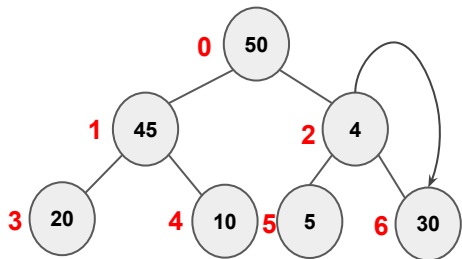
Troca

i=esquerda (1)

Heap

- Remoção (max-heap)

- Sempre da raiz
- Promove o último elemento do vetor como raiz
- Realoca o elemento em um nó em que ele seja menor que o filho



50	45	4	20	10	5	30	4
----	----	---	----	----	---	----	---

i=2

direita= 6

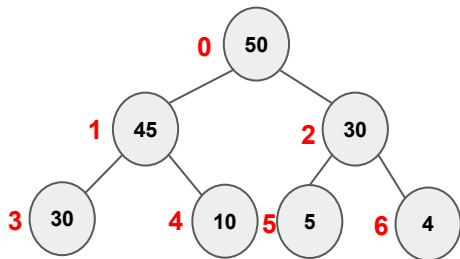
esquerda = 5

Escolhe o maior dos dois

Heap

- Remoção (max-heap)

- Sempre da raiz
- Promove o último elemento do vetor como raiz
- Realoca o elemento em um nó em que ele seja menor que o filho



50	45	30	20	10	5	4	4
----	----	----	----	----	---	---	---

i=2

direita= 6

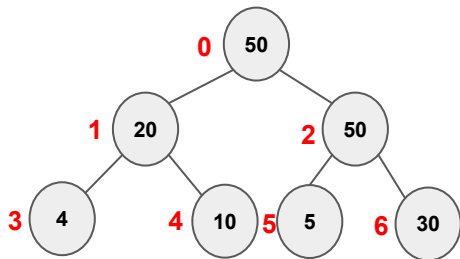
esquerda = 5

Escolhe o maior dos dois

Heap

- Remoção (max-heap)

- Sempre da raiz
- Promove o último elemento do vetor como raiz
- Realoca o elemento em um nó em que ele seja menor que o filho



size = 6

50	45	30	20	10	5	4	4
----	----	----	----	----	---	---	---

FEITO

Heap

- Animação: <https://visualgo.net/en/heap>